

Traffic Crash Prediction

Group 10 - CSCI 5523: Introduction to Data Mining

Rehan Ahmad, Ian Corsiga, Adam Meyers, Jeffrey Qian, & Clayton Sparrow

University of Minnesota

December 2025

INTRODUCTION

Since the year 2000, between 32,000 and 44,000 people have died in a traffic collision every single year in the United States of America [1]. Many more non-fatal, but still costly and life-altering traffic collisions occur each year. While the blame for such collisions is often passed solely onto the vehicle's drivers, they can also be analyzed as a failure in road design, vehicle design, emergency response systems, and license distribution. In this paper, we attempt to create statistical models that can identify and support necessary improvements to road design and emergency response systems.

In the sections below, we lay out our answer to the question: given locational, time, and condition information about previous traffic collisions, can we predict where and when traffic collisions are most likely to occur in the future? To solve this problem, we took state and federal government data on historical traffic collisions and weather conditions throughout the state of Minnesota, further reduced to focus on the Twin Cities metro area, and created two models to analyze this data and predict the times and locations of future traffic collisions. One model utilizes logistic regression, while the other is a simple artificial neural network. Further details on each model are described below.

METHODS

1. Dataset

Two primary data sets were used for the training and testing of each model. The primary dataset used was the Minnesota Crash Data available through the Road Safety Information Center of Minnesota [2]. This data includes one record for every police reported traffic collision within the state of Minnesota since 2016. Each record includes a variety of features, including location, date, time of day, and additional information about the collision such as the number of occupants, whether it was a hit and run, vehicle makes and models, the type and condition of the road, and more.



Figure 1: Minnesota Crash Data

This primary dataset was supplemented with weather data from the National Oceanic and Atmospheric Administration [3]. This dataset records weather and climate conditions from official weather stations throughout the state of Minnesota. Along with locational and time features that were used to combine the two datasets, this dataset also includes features for precipitation, air temperature, and weather type.

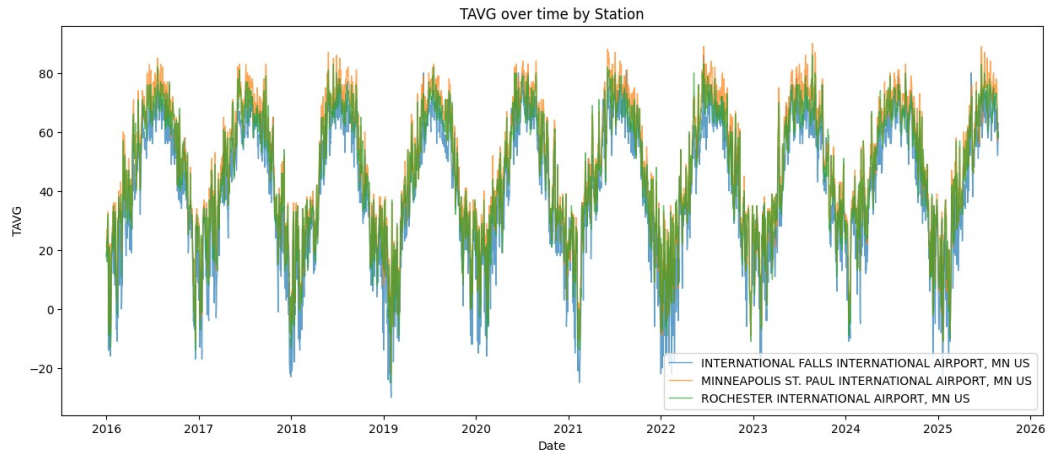


Figure 2: Average Temperature Over Time by Station (2016-2025, 3 Minnesota Stations)

2. Preprocessing

We designed a function to divide a specified area, using latitude and longitude, into a series of square grid cells of variable side lengths, such as 500m by 500m, or 5km by 5km. This function was then used to create a GeoDataFrame with a single record for each n by n grid cell within the maximum and minimum specified latitudes and longitudes. Initially this was set to the maximum and minimum coordinates in the crash data set, but in order to reduce the size of the data, and remove unnecessary records such as those representing sections within surrounding states, this was later specified to a latitude between 44.545707 and 45.681479, and a longitude between -94.390340 and -92.759918. This roughly corresponds to the Twin Cities metropolitan area. As this is the area with the highest occurrences of crashes within Minnesota, this was a logical area to focus on. This also inadvertently filtered out the most notable outliers within the original crash data, as it filtered out the Northeastern tip of Minnesota, which had been marked with the most crashes of anywhere in the entire state, almost certainly due to being some sort of default locational value for any record that did not have a specified location.

This GeoDataFrame was then expanded to create a daily record for each grid cell, using a specified start and end date, neither of which can be outside of the earliest and latest dates in the crash data. Every single recorded crash was then assigned to one daily cell based on its location and date, keeping a count of crashes (`num_crashes`) and a binary classification (`crash_tomorrow`) indicating whether a crash occurred the following day in the same cell. Additional features from the crash and weather datasets were assigned to each cell depending on whether or not they were deemed to be useful. Further detail on these specific features are described below in the feature selection section. Finally, the final daily record for each cell was removed, as `crash_tomorrow` was always set to 0 for these cells, due to a lack of crash data for the following day.

Two datasets were generated using this process. A training dataset was generated, using data from January 1st, 2016 to December 31st, 2023. The remaining data from January 1st, 2024 to September 30th, 2025 was used as a testing dataset.

3. Exploratory Analysis

Initial analysis revealed that the data is highly imbalanced and sparse, with most cells on most days experiencing zero crashes. There were also some clear patterns noticed when it came to the time and location features, even without any model training. For one, it was noted that crashes occurred more frequently on weekdays rather than weekends. This observation makes a lot of intuitive sense, as many people travel to and from work on the weekdays, but don't need to travel on weekends, and thus there would be fewer crashes overall on weekends, simply due to less cars being on the road.

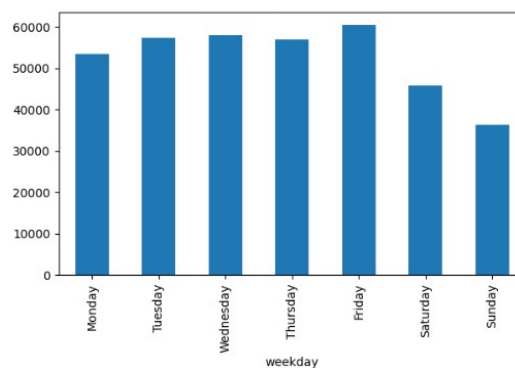


Figure 3: Total Crashes by Day of the Week (2017-2024)

Zooming to a larger time scale, we also noticed some patterns over the course of the entire year, with certain months logging significantly more traffic collisions than others. The winter months of December and January logged more crashes than any other month of the year, which is very sensible as most Minnesota winters have frequent snowfalls in December and January, which leads to an increase in the difficulty of driving, biking, and walking. April, on the other hand, was the least crash prone month of the year. This is harder to explain, as we see little immediate reason why less crashes would occur in April compared to other months with less inclement weather.

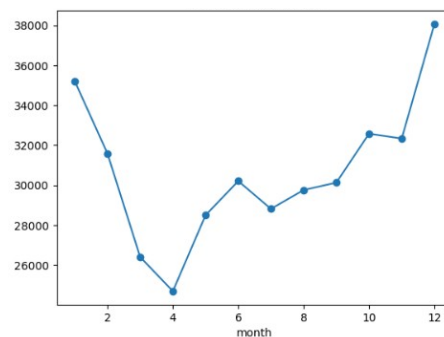


Figure 4: Total Crashes per Month of the Year (2017-2024)

There were also notable patterns when the data was plotted geometrically, showing which grid cells had the highest number of crashes. Across the entirety of Minnesota, collisions were most

densely clustered in the Twin Cities metropolitan area, along with smaller spikes around St. Cloud, Duluth, Rochester, and Mankato. When reduced to just the Twin Cities metropolitan area, we can identify separate spikes around the downtowns of Minneapolis and St. Paul. These areas are more densely populated, resulting in a higher number of cars on the road. In addition, unlike highways, which also feature high car traffic, intersections and close contact with bikers and pedestrians can also likely lead to higher rates of collisions.

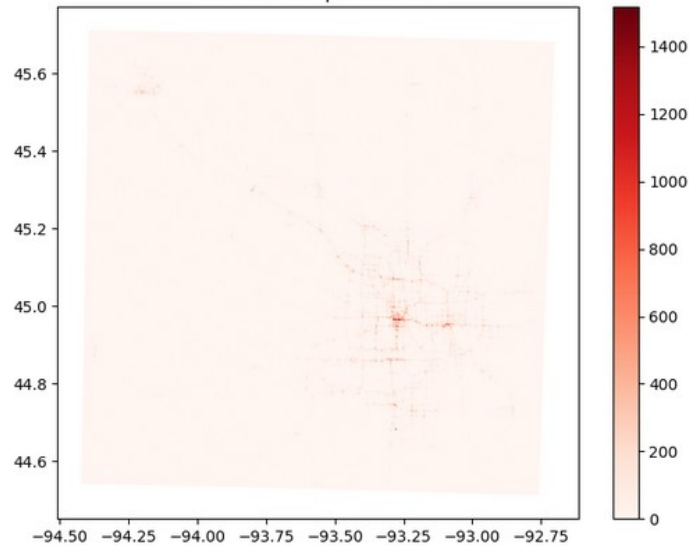


Figure 5: Total Crashes per Grid Cell (2017-2024, Twin Cities Metropolitan Area)

While the highways did feature less density of crashes compared to urban areas, we can still see that major roadways such as Interstate 35 and 94 also have noticeable outlines when plotted, especially near major cities like Minneapolis and St. Paul. Just like with the downtown areas, this can likely be attributed to the high numbers of cars that travel these major roadways.

4. Feature Selection

We initially had a crash dataset containing ~ 40 traffic-related features and narrowed the list down to retain the most relevant features in forecasting whether a crash will occur at a given location on the following day. Irrelevant or overly specific variables (e.g. VehicleMakeModel, Zipcode, Num_Occupants) were dropped because they add noise without providing predictive power. We prioritized features that pertained to time, location, and environment (e.g. Day, Weather/Surface Conditions). From the weather dataset, we retained features like temperature (TAVG), precipitation (PRCP), snowfall (SNOW), and WT code indicators since these features can be used to explain crash likelihood.

5. Model: Logistic Regression

5.1 Setup

We start by discretizing the study area into a regular grid. For each day t and cell c we have a crash count:

- $Y[t, c]$ = number of crashes in cell c on day t .

Stacking all days and cells gives us a matrix:

- Y has shape (T, C) , where T is the number of days and C is the number of cells.

Our prediction task is:

- For each pair (t, c) , predict whether there will be at least one crash in cell c on day $t + 1$.

We encode this as a binary label:

- $y[t, c] = 1$ if $Y[t + 1, c] \geq 1$,
- $y[t, c] = 0$ otherwise.

The model outputs a probability $p[t, c]$ that there will be at least one crash tomorrow in that cell, based only on information available up to day t .

5.2 Hawkes-Style Temporal Features

Crashes are not independent over time. For example, if a crash happens at a location, that location might be “hot” for a little while (temporary hazard, ongoing disruption, etc.), and that extra risk should fade over time. This is the same kind of idea behind Hawkes processes (self-exciting point processes). Instead of fitting a full Hawkes model, we build simple exponential-decay features that capture this behavior in discrete time.

For each cell c and each time scale τ (in days), we maintain an intensity-like state $s_{\tau}[t, c]$ using a recurrence:

- $s_{\tau}[t, c] = \rho * s_{\tau}[t - 1, c] + Y[t - 1, c]$
- $\rho = \exp(-1 / \tau)$
- $s_{\tau}[0, c] = 0$.

Where:

- $Y[t - 1, c]$ is yesterday’s crash count in that cell.
- $s_{\tau}[t, c]$ is a running summary of all earlier crashes, but with exponential decay: recent crashes matter more, older ones slowly “fade out”.

A single time scale is not enough. If τ is too small, we only see very short-lived spikes. If τ is too large, we smooth everything so much that short-term bursts disappear. To fix this, we:

- Choose a set of time scales, for example $(3, 7, 30)$ days.
- Compute a separate $s_{\tau}[t, c]$ for each τ in that set.

So for each (t, c) we end up with a feature vector like:

- $x[t, c] = (s_3[t, c], s_7[t, c], s_{30}[t, c], \dots)$.

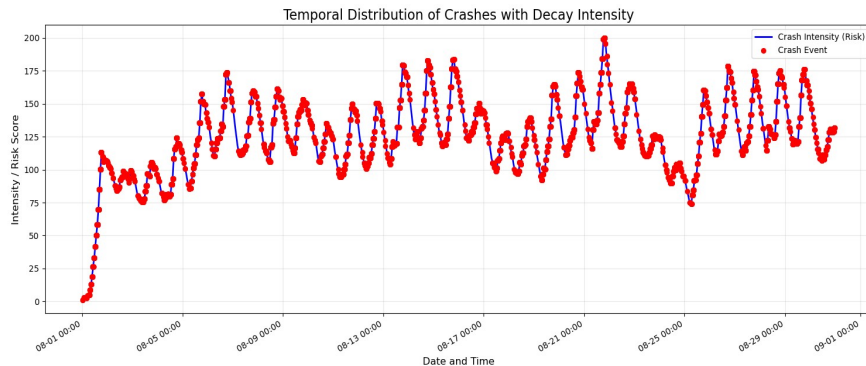


Figure 6: Hawkes Process Crash Occurrences and Crash Probability (Aug 2025)

All temporal features are computed by `compute_hawkes_features`, and the final feature matrix and label vector are built by `build_training_data_from_counts`. The data-to-features pipeline looks like this:

1. Start with Y of shape (T, C) , where $Y[t, c]$ is crash count for day t , cell c .
2. Fix a set of time scales $\text{taus} = (\text{tau}_1, \dots, \text{tau}_D)$.
3. For each tau and each (t, c) , compute $s_{\text{tau}}[t, c]$ from past counts only (no future leakage).
4. For each $t = 0, \dots, T - 2$ and each cell c :
 - Features: $x[t, c] = (s_{\text{tau}_1}[t, c], \dots, s_{\text{tau}_D}[t, c])$.
 - Label: $y[t, c] = 1$ if $Y[t + 1, c] \geq 1$, otherwise 0.
5. Stack all (t, c) into:
 - X with shape $((T - 1) * C, D)$,
 - y with shape $((T - 1) * C,)$.

In the run described here, Y is built from `sample_crash_data.csv` using a 10 km grid over the study area. The grid is created by `generate_grid`, and daily cell-level counts come from `create_cell_day_df` in `src/preprocessing.py`. The sample data covers 2025-08-01 to 2025-08-29, and the label rate is about 0.9%, which means crashes are quite rare at this resolution.

5.3 L2-Regularized Logistic Regression

Once we have X and y , we use a fairly standard logistic regression model with L2 regularization. For each row i (which corresponds to some (t, c) pair), we have:

- Input feature vector: $x_i \in \mathbb{R}^D$
- Binary label: $y_i \in \{0, 1\}$

The model is:

- $z_i = w^T x_i + b$
- $p_i = 1 / (1 + \exp(-z_i))$

where:

- w is a weight vector in $\mathbb{R}^D$,
- b is a scalar bias,
- p_i is interpreted as the probability of at least one crash in that cell the next day.

Training objective:

We train by minimizing a weighted logistic loss with an L2 penalty:

- $L(w, b) = (1 / n) * \sum_i w_i(s) * [-y_i \log p_i - (1 - y_i) \log(1 - p_i)] + (\text{lambda} / 2) * ||w||^2$

where:

- n is the number of training examples,
- lambda is the L2 regularization strength,
- $w_i(s)$ is an optional sample weight for example i .

The sample weights are used to deal with class imbalance. When weights are enabled, positives can be up-weighted relative to negatives, for example:

- $w_i(s) = \text{pos_weight}$ if $y_i = 1$,
- $w_i(s) = 1$ if $y_i = 0$,

with $\text{pos_weight} \approx (\# \text{ negatives}) / (\# \text{ positives})$.

The model is optimized using gradient descent with:

- a fixed learning rate lr , and
- a fixed number of epochs (passes over the data).

5.4 Hyperparameter Search

For hyperparameters we run a small grid search. The idea is to try a handful of reasonable settings and pick what works best in cross-validation.

The search space includes:

- Time scales for Hawkes features:
 - tau sets: (3, 7, 30), (7, 14, 30), (7, 30, 60)
- Optimization and regularization:
 - learning rate lr : 0.01, 0.05
 - L2 strength λ : $1e-3$, $1e-2$, $1e-1$
- Class weighting and training length:
 - `pos_weight`: None or 5
 - epochs: 50 or 100

For each combination of these hyperparameters, we:

1. Build X and y from the crash data using the Hawkes-feature pipeline.
2. Run 3-fold stratified cross-validation on the rows of (X, y) (we keep the positive/negative ratio roughly the same in each fold).
3. Compute the following metrics on each fold:
 - log-loss (cross-entropy),
 - ROC-AUC,
 - average precision (area under the PR curve).
4. Average the metrics across folds and rank configurations based on high ROC-AUC / average precision and low log-loss. We fix a random seed so the results are reproducible.

On `sample_crash_data.csv` with a 10 km grid, the best configuration we found was:

tau set	(3, 7, 30)
learning rate, lr	0.05
L2: λ	$1e^{-3}$
<code>pos_weight</code>	None
epochs	100

Table 1: Hyperparameter Configuration

This setup achieved approximately:

log-loss	0.239
-----------------	-------

ROC-AUC	0.85
Average precision	0.508

Table 2: Logistic Regression Validation Measures

5.5 Limitations

The cross-validation we use in the hyperparameter search is not time-aware. It treats rows of (X, y) as if they were independent and identically distributed. A stricter evaluation would train on earlier dates and test on later dates to simulate real deployment. In addition, Results are tied to the current dataset and grid resolution (10 km cells, August 2025 sample). Different grid sizes, longer time spans, or more complete crash data could change the performance and might require re-tuning. Finally, the dataset is heavily imbalanced. We only lightly touched class weighting and thresholding. There is room to explore better ways to handle imbalance (e.g., different loss functions, cost-sensitive thresholds, or rank-based evaluation).

5.6 Next Steps

In the future, the Hawkes-style features can be combined with additional features (weather, road type, neighboring cells, etc.). Additionally, the logistic regression baseline could be swapped out for more flexible models (tree ensembles, spatio-temporal neural nets) while keeping the same feature pipeline. We could also reuse the same code for other cities or regions by just changing the input data and grid settings.

6. Model: Artificial Neural Network

The second model we implemented was a simple artificial neural network (ANN) designed to provide a nonlinear alternative to the logistic regression model while using the same underlying feature representation. Unlike the logistic regression model, which explicitly assumes a linear relationship between the features and the log-odds of a crash, the ANN can learn more complex interactions between temporal, spatial, and environmental variables.

To ensure a fair comparison between models, the ANN was trained on the same dataset as the logistic regression model, though the Hawkes process was not used to generate additional features. The ANN was implemented as a feedforward multilayer perceptron with two hidden layers (64 and 32 neurons) and ReLU activation, producing a probability of a crash occurring the following day. All features were standardized prior to training.

The ANN model was trained on all Twin Cities Metropolitan Area crash data from 2016 to 2023, and tested using crash data from 2024 to September 2025. Seven validation measures were taken based on this performance, which are shown below.

Measure	Result (ANN Model)
---------	--------------------

Brier Score	0.049809
Log-Loss	0.198600
PR-AUC	0.112763
ROC-AUC	0.689069
Hit-Rate at 500	0.168000
Lift at 500	3.11
ECE (10 bins)	0.0051

Table 3: ANN Model Validation Measures

There are a few promising results in these validation measures. For one, a ROC-AUC of ~ 0.689 implies the model has some success discriminating between crash and non-crash cells. Additionally, PR-AUC and Hit-Rate at 500 show substantially higher values than the base rate of crash_tomorrow at around $\sim 5\text{-}6\%$. The lift at 500 additionally implies that selecting cells using this model is 3.11 times more effective than selecting cells at random. Low values for the brier score, log-loss, and ECE indicate that the predicted probabilities are reliable and that the model may be suitable for use in predicting crash risk within cities. The reliability plot below further backs up the model, showing that when the model predicts crash risks between 0.0 to ~ 0.2 , crashes end up happening with about the same frequency. This reliability falls off with higher probabilities, but that is expected as crashes have very low probability, and the model very infrequently gives probabilities higher than ~ 0.25 .

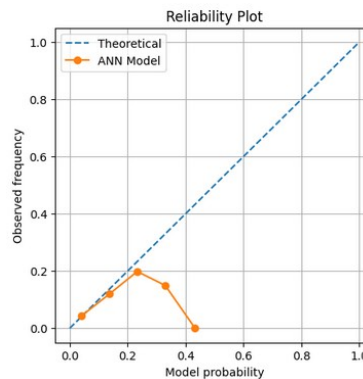


Figure 7: ANN Model Reliability Plot

CONCLUSIONS & FUTURE WORK

Predicting crash risk on a spatial and temporal scale presented a challenge due to differences in roadways, travel patterns, and reporting across areas. We created two models to predict crash risk over the Twin Cities Metropolitan area with moderate levels of success. Between the two models, the neural network model showed better measures of reliability, though the logistic regression model also showed promising results, and both models could likely be improved on with additional time.

There are several approaches we could take to improve the performance of our models. First, additional traffic-related data can be incorporated into the dataset to better understand day-to-day variability in roadway usage. Metrics from the Annual Average Daily Traffic (AADT) that measure congestion and traffic could help differentiate locations that are at higher risk due to the amount of vehicles that pass through. We could also do more feature engineering to include holidays or periods of seasonal vacation to capture temporary surges in traffic that can be used to predict the likelihood of accidents occurring. Furthermore, using previous days' weather, traffic, and crash information to construct short-term temporal features, like rolling averages or cumulative totals over n days, could help show conditions that increase risk leading into the following day. Besides feature engineering, we could also explore different models and strategies, especially given the imbalanced nature of crash outcomes. Gradient-boosted tree methods, such as XGBoost, may be able to capture complex relationships and outperform current models. We could also apply sampling techniques to address the class imbalance, such as undersampling or SMOTE, to improve model performance.

For future endeavors, we could expand on this project in many various ways. For one, we can increase the temporal scope of the dataset by incorporating additional years of traffic data to improve model robustness. We could also extend the analysis beyond the current area to include other regions or the state of Minnesota as a whole. This would allow us to compare region-specific models and a statewide model to assess geographic variability in model performance and predictive patterns. Another approach is to generate separate models tailored to specific scenarios, such as winter-only or urban regions only. Exclusively analyzing crashes in winter conditions could result in higher model performance as we aim to limit other confounding variables.

SOURCE CODE

Jupyter notebooks and python files used for this project can be found at <https://github.com/asbmeyers/CSCI-5523-Traffic-Crash-Prediction>.

REFERENCES

- [1] FARS Encyclopedia: National Highway Traffic Safety Administration. <https://www-fars.nhtsa.dot.gov/Main/index.aspx> (accessed Dec 10, 2025).
- [2] "Crash Data." Minnesota Road Safety Information Center. <https://roadsafetyinfocenter.mn.gov/map/information/crashes> (accessed Oct 28, 2025).
- [3] "Climate Data Online." National Centers for Environmental Information: National Oceanic and Atmospheric Administration. <https://www.ncei.noaa.gov/cdo-web/search?datasetid=GHCND> (accessed Nov 14, 2025).